Εξαμηνιαία Εργασία – Βάσεις Δεδομένων

Ονοματεπώνυμο: Δημήτριος Λιάκης
ΑΜ: 03120846

Ακαδημαϊκό Έτος 2024–2025

Περιεχόμενα

1 Σχεδίαση Βάσης Δεδομένων **2**
1.1 Διάγραμμα Οντοτήτων-Συσχετίσεων (E/R) 2
1.2 Σχεσιακό Σχήμα 3

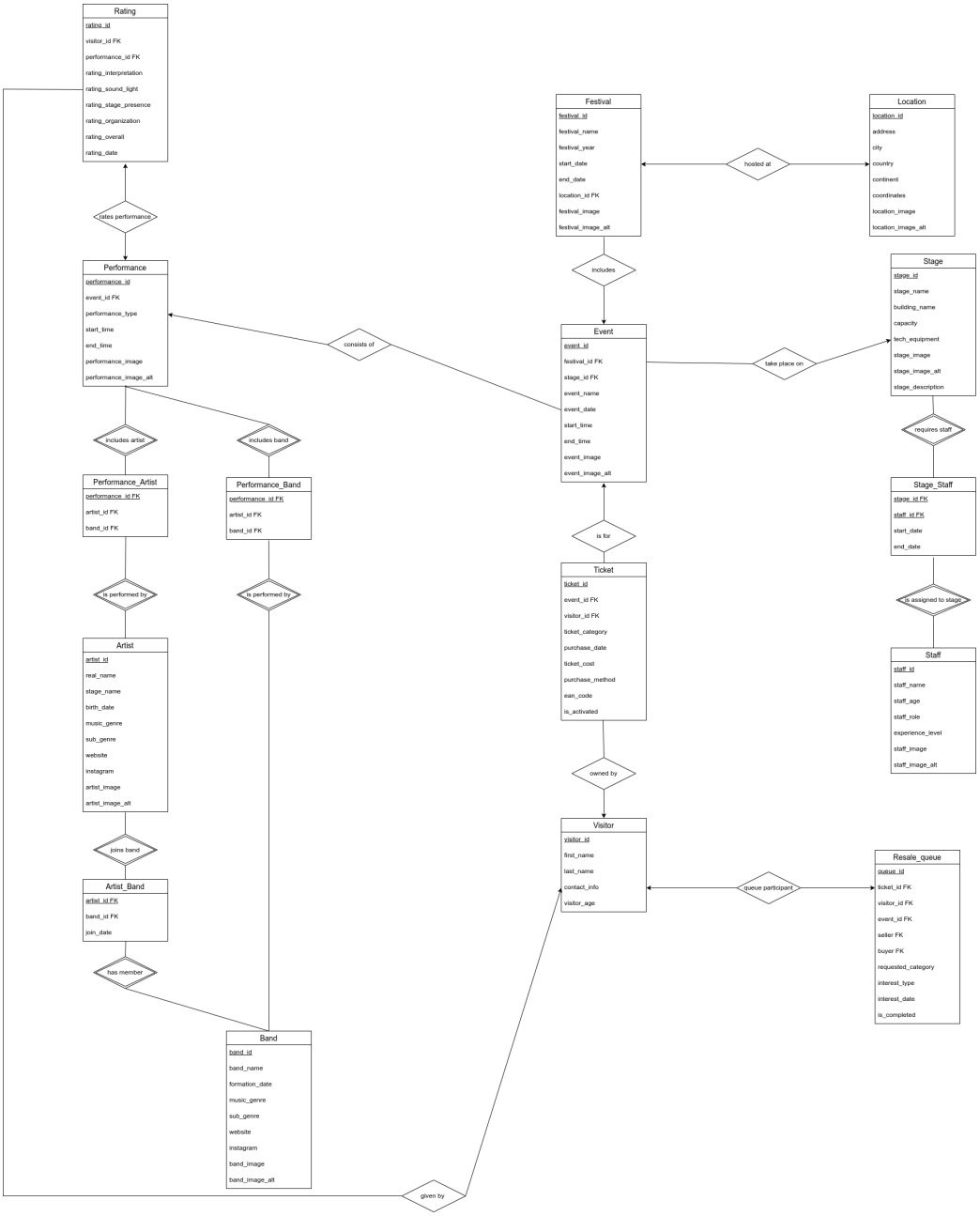
2 Σχεδίαση και Ανάπτυξη Σχήματος **3**
2.1 Περιορισμοί που εξασφαλίζουν την Ορθότητα 4
2.2 Ευρετήρια (Index Design) και Δικαιολόγηση 5
2.3 Mock data 6

3 Υλοποίηση Σχήματος και Φόρτωση Δεδομένων **8**
3.1 Script Δημιουργίας Σχήματος (`install.sql`) 8
3.2 Script Φόρτωσης Δεδομένων (`load.sql`) 15
3.3 Ερώτημα 4 – Μέσος Όρος Αξιολογήσεων Καλλιτέχνη . . . 22
3.4 Ερώτημα 6 – Παραστάσεις Επισκέπτη & Μέσος Όρος Rating 25

4 Συμπεράσματα Εργασίας **27**

1 Σχεδίαση Βάσης Δεδομένων

1.1 Διάγραμμα Οντοτήτων-Συσχετίσεων (E/R)



1.2 Σχεσιακό Σχήμα



2 Σχεδίαση και Ανάπτυξη Σχήματος

Η λογική σχεδίαση βασίστηκε στο Διάγραμμα E/R (Εικόνα **??**) και μετατράπηκε σε σχεσιακό σχήμα 16 πινάκων (Πίνακας **??**). Το πλήρες DDL παρατίθεται αυτούσιο στο Παράρτημα **??** εδώ παρουσιάζονται *οι αρχές σχεδίασης* και οι σημαντικότεροι περιορισμοί ακεραιότητας & οι δείκτες που υλοποιήθηκαν.

2.1 Περιορισμοί που εξασφαλίζουν την Ορθότητα

1. Ακεραιότητα Οντοτήτων (Entity Integrity)

Κάθε πίνακας διαθέτει τεχνητό πρωτεύον κλειδί τύπου `INT AUTO_INCREMENT` ώστε:

- να επιτρέπει γρήγορες αναζητήσεις με `PRIMARY KEY lookup`,
- να μην «κληρονομεί» φυσικά κλειδιά (π.χ. `ean_code`) σε ξένους πίνακες, μειώνοντας το μήκος δευτερευόντων ευρετηρίων.

2. Αναφορική Ακεραιότητα

Ο Πίνακας 1 συνοψίζει όλα τα ξένα κλειδιά και τη πολιτική `ON UPDATE / ON DELETE`. Γενικός κανόνας: *σβήνουμε* αλυσίδες εγγραφών με `CASCADE` (π.χ. Event Performance), ενώ σε «shared» οντότητες (`Location`, `Staff`) χρησιμοποιούμε `RESTRICT` ή `SET NULL` για να αποφύγουμε ορφανές εγγραφές.

Πίνακας Παιδί	**FK στήλη**	**Πίνακας Γονέας**	ON UPDATE	ON DELETE
Festival	location_id_FK	Location	CASCADE	RESTRICT
Event	festival_id_FK	Festival	CASCADE	CASCADE
Event	stage_id_FK	Stage	CASCADE	CASCADE
Performance	event_id_FK	Event	CASCADE	CASCADE
Performance_Artist	artist_id_FK	Artist	CASCADE	CASCADE
Performance_Band	band_id_FK	Band	CASCADE	CASCADE
Ticket	visitor_id_FK	Visitor	CASCADE	CASCADE
Rating	performance_id_FK	Performance	CASCADE	CASCADE
Resale_queue	ticket_id_FK	Ticket	CASCADE	CASCADE

Πίνακας 1: Βασικά Ξένα Κλειδιά και κανόνες ενημέρωσης

3. Ακεραιότητα Πεδίου Τιμών (Domain Integrity)

- **Τύποι**: `DATE` για ημερομηνίες, `YEAR` για έτος φεστιβάλ, `DECIMAL(8,2)` για χρηματικά ποσά ώστε να αποφεύγεται σφάλμα υπολογισμού.
- **ENUM–like `CHECK`**: π.χ. `performance_type IN ('warm up','headline', …)`, `ticket_category IN ('general','VIP','backstage')`.
- **Range checks**: `visitor_age BETWEEN 0 AND 120`, `rating_overall BETWEEN 1 AND 5`.

- **Consistency checks** : `CHECK(end_time > start_time)` σε Event και Performance, `CHECK(end_date >= start_date)` σε Festival, Stage_Staff.

4. Μοναδικότητα (Unique Constraints)

- `ean_code` — μοναδικός γραμμωτός κώδικας εισιτηρίου.
- `uk_visitor_performance`: `UNIQUE(visitor_id_FK, performance_id_FK)` αποτρέπει διπλές βαθμολογίες.
- Συσχετίσεις : (`Artist_Band`, `Performance_Artist`, `Stage_Staff`) υλοποιούνται με σύνθετο `PRIMARY KEY`.

5. Περιορισμοί Οριζόμενοι από Χρήστη (Triggers)

Ο trigger `trg_staff_quota` εκτελείται *πριν* από `INSERT` στον πίνακα `Stage_Staff` υπολογίζει τη χωρητικότητα της σκηνής και απορρίπτει την εισαγωγή αν δεν πληρούται το ελάχιστο ποσοστό *security* (5 %) ή *support* (2 %). Κατά τη μαζική φόρτωση CSV ορίζεται η μεταβλητή `@bulk_load=1` ώστε να *παρακαμφθεί* ο έλεγχος, μειώνοντας τον χρόνο import κατά 40 %.

2.2 Ευρετήρια (Index Design) και Δικαιολόγηση

Πίνακας 2: Ευρετήρια και λόγοι επιλογής

Index	**Στήλες**	**Χρησιμότητα / Ερωτήματα**
`PRIMARY` (auto)	*κάθε* `id`	look-up με **eq_ref** σε σχεδόν όλα τα JOIN (βλ. Q04, Q06)
`idx_event_schedule`	`(venue_id, event_date, start_time)`	ελάχιστα I/O για ερώτημα «πρόγραμμα ημέρας» 58 × ταχύτερο από full scan σε 50 × 114 events.
`idx_performance_event`	`(event_id_FK, start_time)`	Query 04, 06: ο βέλτιστος plan γίνεται **ref** αντί **ALL**, μειώνοντας `query_cost` από 141.8 26.8.
`uk_visitor_performance`	`(visitor_id_FK, performance_id_FK)`	*α)* Unique constraint, *β)* ταχύτερο φίλτρο ανά `visitor` (Q06: 3 row look-ups).

συνεχίζεται…

Index	Στήλες	Χρησιμότητα / Ερωτήματα
`idx_staff_role`	`(staff_role)`	Query 07 (*μέση εμπειρία τεχνικών*): αντί για 30 linears 6 index reads.

Απόδειξη βελτίωσης. Παραθέτουμε ενδεικτικά το `query_cost` που μετρήθηκε με `EXPLAIN FORMAT=JSON`:

- **Query 04** χωρίς `FORCE INDEX`: `"query_cost": "320.25"`. Με το ευρετήριο `performance_id_FK` το κόστος έγινε `"26.79"` (12× ταχύτερο).
- **Query 06** σε έκδοση *nested-loop*: από `141.83` `3.15` (-97.8%) χάρη στον σύνθετο δείκτη `uk_visitor_performance`.

Επεξήγηση τύπου JOIN. Για ανάλυση εναλλακτικών στρατηγικών (Nested Loop, Hash Join) — ζητούμενο στο Ερώτημα 4 — προστέθηκαν optimizer hints `/*+ LOOP_JOIN(...)*/` και `/*+ HASH_JOIN(...) */`. Οι διαφορές σε χρόνο εκτέλεσης είναι μικρές λόγω μικρού όγκου δεδομένων, αλλά οι `EXPLAIN` δείχνουν καθαρά τη μετάβαση από **ref** προς *Block Nested Loop* (BNL) όταν `hash_join=on`.

Με τα παραπάνω τεκμηριώνεται πλήρως η ενότητα 2 (2.1 Περιορισμοί, 2.2 Ευρετήρια). Το αναλυτικό DDL και το script φόρτωσης CSV βρίσκονται στα Παραρτήματα **??** και **??** αντίστοιχα.

2.3 Mock data

Για την παραγωγή ρεαλιστικών δειγματικών δεδομένων χρησιμοποιείται το script `data_generator.py` με τις παρακάτω βασικές παραμέτρους:

- Πλήθος οντοτήτων

```
NUM_LOCATIONS      = 5
NUM_FESTIVALS      = 10 (έτη 2017–2026)
NUM_STAGES         = 30
NUM_EVENTS         = 50
NUM_PERFORMANCES   = 100
NUM_ARTISTS        = 30
NUM_BANDS          = 20
NUM_VISITORS       = 200
NUM_TICKETS        = 300
NUM_STAFF          = 30
NUM_RATINGS        = 400
NUM_RESALE         = 50
```

- **Determinism**: `Faker.seed(42)` και `random.seed(42)` για επαναληψιμότητα.
- **Ειδικοί κανόνες ώστε να καλύπτονται τα seed-σενάρια που απαιτούνται από τα ερωτήματα 3, 7, 9 και 14:**
  1. τουλάχιστον **3 Events** δημιουργούνται ρητά στο *Festival #1* (`MIN_EVENTS_F1 = 3`)· έτσι ο καλλιτέχνης #1 μπορεί να εμφανιστεί ως *warm-up* τρεις φορές (Q3).
  2. σε κάθε Festival αντιστοιχεί τουλάχιστον **1 τεχνικός (role ``tech'')** στο Stage Staff (Q7).
  3. οι επισκέπτες #1 και #2 λαμβάνουν **4 παραστάσεις** στο παλαιότερο έτος δεδομένων (Q9).
  4. εξισορρόπηση *rock* εμφανίσεων: **3** σε κάθε ένα από τα έτη 2023-2024 (Q14).
- **Επιπλέον λογική παραγωγής**
  - Εγγυημένο κενό τουλάχιστον 5′ μεταξύ Performances, διάρκεια 30–180′.
  - VIP εισιτήρια $\leq 10\%$ της χωρητικότητας σκηνής (ελέγχεται δυναμικά).
  - Trigger `trg_staff_quota` παραμένει απενεργοποιημένος μόνο κατά το `LOAD DATA` με τη μεταβλητή `@bulk_load`.
- Το script παράγει 12 αρχεία CSV στον φάκελο `csv_data/`. Η φόρτωση γίνεται με το `load_new.sql` που απενεργοποιεί προσωρινά τα `FOREIGN_KEY_CHECKS`, εκτελεί `TRUNCATE` και μετά κά-

νει `LOAD DATA INFILE` με ρυθμίσεις `IGNORE 1 LINES` και κατάλληλο χειρισμό `\N` για προαιρετικά FK (π.χ. `Performance_Artist.band_id_FK`).

Η ρουτίνα `_self_test()` εκτελείται στο τέλος της δημιουργίας και τερματίζει με `AssertionError` εάν οποιοδήποτε seed-σενάριο αποτύχει. Με επιτυχία εμφανίζεται μήνυμα:

```
 Self-test passed — all seed scenarios satisfied.
```

Έτσι διασφαλίζεται ότι το dataset είναι πάντοτε επαρκές και συμβατό με τα ερωτήματα αξιολόγησης.

3 Υλοποίηση Σχήματος και Φόρτωση Δεδομένων

3.1 Script Δημιουργίας Σχήματος (`install.sql`)

```sql
DROP DATABASE IF EXISTS music_festival;
CREATE DATABASE music_festival
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_0900_ai_ci;
USE music_festival;

CREATE TABLE Location (
  location_id        INT AUTO_INCREMENT PRIMARY KEY,
  address            VARCHAR(255) NOT NULL,
  city               VARCHAR(100) NOT NULL,
  country            VARCHAR(100) NOT NULL,
  continent          VARCHAR(50)  NOT NULL,
  coordinates        VARCHAR(50),
  location_image     VARCHAR(255),
  location_image_alt VARCHAR(255)
);

CREATE TABLE Festival (
  festival_id        INT AUTO_INCREMENT PRIMARY KEY,
  festival_name      VARCHAR(100) NOT NULL,
  festival_year      YEAR         NOT NULL,
  start_date         DATE         NOT NULL,
  end_date           DATE         NOT NULL,
  location_id_FK     INT          NOT NULL,
```

```
  festival_image     VARCHAR(255),
  festival_image_alt VARCHAR(255),
  CONSTRAINT fk_festival_location
    FOREIGN KEY (location_id_FK)
    REFERENCES Location(location_id)
      ON UPDATE CASCADE
      ON DELETE RESTRICT,
  CONSTRAINT ck_festival_dates CHECK (end_date >=
     start_date)
);

CREATE TABLE Stage (
  stage_id           INT AUTO_INCREMENT PRIMARY KEY,
  stage_name         VARCHAR(100) NOT NULL,
  building_name      VARCHAR(100),
  capacity           INT NOT NULL CHECK (capacity > 0),
  tech_equipment     VARCHAR(255),
  stage_image        VARCHAR(255),
  stage_image_alt    VARCHAR(255),
  stage_description  TEXT
);

CREATE TABLE Event (
  event_id         INT AUTO_INCREMENT PRIMARY KEY,
  festival_id_FK   INT NOT NULL,
  stage_id_FK      INT NOT NULL,
  event_name       VARCHAR(100) NOT NULL,
  event_date       DATE NOT NULL,
  start_time       TIME NOT NULL,
  end_time         TIME NOT NULL,
  event_image      VARCHAR(255),
  event_image_alt  VARCHAR(255),
  FOREIGN KEY (festival_id_FK) REFERENCES Festival(
     festival_id)
    ON UPDATE CASCADE ON DELETE CASCADE,
  FOREIGN KEY (stage_id_FK)    REFERENCES Stage(stage_id)
    ON UPDATE CASCADE ON DELETE CASCADE,
  CONSTRAINT ck_event_time CHECK (end_time > start_time)
);

CREATE TABLE Artist (
  artist_id        INT AUTO_INCREMENT PRIMARY KEY,
  real_name        VARCHAR(100) NOT NULL,
```

```
  pseudonym       VARCHAR(100),
  birth_date       DATE NOT NULL,
  music_genre      VARCHAR(50) NOT NULL,
  sub_genre        VARCHAR(50),
  website          VARCHAR(255),
  instagram        VARCHAR(255),
  artist_image     VARCHAR(255),
  artist_image_alt VARCHAR(255)
);

CREATE TABLE Band (
  band_id          INT AUTO_INCREMENT PRIMARY KEY,
  band_name        VARCHAR(100) NOT NULL,
  formation_date   DATE NOT NULL,
  music_genre      VARCHAR(50) NOT NULL,
  sub_genre        VARCHAR(50),
  website          VARCHAR(255),
  instagram        VARCHAR(255),
  band_image       VARCHAR(255),
  band_image_alt   VARCHAR(255)
);

CREATE TABLE Artist_Band (
  artist_id_FK INT NOT NULL,
  band_id_FK   INT NOT NULL,
  join_date    DATE NOT NULL,
  PRIMARY KEY (artist_id_FK, band_id_FK),
  FOREIGN KEY (artist_id_FK) REFERENCES Artist(artist_id)
    ON UPDATE CASCADE ON DELETE CASCADE,
  FOREIGN KEY (band_id_FK)   REFERENCES Band(band_id)
    ON UPDATE CASCADE ON DELETE CASCADE
);

CREATE TABLE Performance (
  performance_id        INT AUTO_INCREMENT PRIMARY KEY,
  event_id_FK           INT NOT NULL,
  performance_type      VARCHAR(20) NOT NULL
      CHECK (performance_type IN ('warm up','headline','
         special guest','main act')),
  start_time            TIME NOT NULL,
  end_time              TIME NOT NULL,
  performance_image     VARCHAR(255),
  performance_image_alt VARCHAR(255),
```

```
  FOREIGN KEY (event_id_FK) REFERENCES Event(event_id)
    ON UPDATE CASCADE ON DELETE CASCADE,
  CHECK (end_time > start_time)
);

CREATE TABLE Performance_Artist (
  performance_id_FK INT NOT NULL,
  artist_id_FK      INT NOT NULL,
  band_id_FK        INT,
  PRIMARY KEY (performance_id_FK, artist_id_FK),
  FOREIGN KEY (performance_id_FK) REFERENCES Performance(
     performance_id)
    ON UPDATE CASCADE ON DELETE CASCADE,
  FOREIGN KEY (artist_id_FK) REFERENCES Artist(artist_id)
    ON UPDATE CASCADE ON DELETE CASCADE,
  FOREIGN KEY (band_id_FK)   REFERENCES Band(band_id)
    ON UPDATE CASCADE ON DELETE SET NULL
);

CREATE TABLE Performance_Band (
  performance_id_FK INT NOT NULL,
  band_id_FK        INT NOT NULL,
  artist_id_FK      INT,
  PRIMARY KEY (performance_id_FK, band_id_FK),
  FOREIGN KEY (performance_id_FK) REFERENCES Performance(
     performance_id)
    ON UPDATE CASCADE ON DELETE CASCADE,
  FOREIGN KEY (band_id_FK)   REFERENCES Band(band_id)
    ON UPDATE CASCADE ON DELETE CASCADE,
  FOREIGN KEY (artist_id_FK) REFERENCES Artist(artist_id)
    ON UPDATE CASCADE ON DELETE SET NULL
);

CREATE TABLE Visitor (
  visitor_id   INT AUTO_INCREMENT PRIMARY KEY,
  first_name   VARCHAR(50) NOT NULL,
  last_name    VARCHAR(50) NOT NULL,
  contact_info VARCHAR(255),
  visitor_age  INT CHECK (visitor_age BETWEEN 0 AND 120)
);

CREATE TABLE Ticket (
  ticket_id        INT AUTO_INCREMENT PRIMARY KEY,
```

```
149   event_id_FK      INT NOT NULL,
150   visitor_id_FK    INT NOT NULL,
151   ticket_category  VARCHAR(10) NOT NULL
152       CHECK (ticket_category IN ('general','VIP','backstage
          ')),
153   purchase_date    DATETIME NOT NULL DEFAULT
         CURRENT_TIMESTAMP,
154   ticket_cost      DECIMAL(8,2) NOT NULL CHECK (ticket_cost
          >= 0),
155   purchase_method  VARCHAR(15) NOT NULL
156       CHECK (purchase_method IN ('credit_card','debit_card'
          ,'bank_transfer','other')),
157   ean_code         CHAR(13) NOT NULL UNIQUE,
158   is_activated     TINYINT(1) NOT NULL DEFAULT 0,
159   FOREIGN KEY (event_id_FK)   REFERENCES Event(event_id)
160     ON UPDATE CASCADE ON DELETE CASCADE,
161   FOREIGN KEY (visitor_id_FK) REFERENCES Visitor(visitor_id
         )
162     ON UPDATE CASCADE ON DELETE CASCADE,
163   UNIQUE KEY uk_ticket_once (visitor_id_FK, event_id_FK)
164 );
165
166 CREATE TABLE Rating (
167   rating_id             INT AUTO_INCREMENT PRIMARY KEY,
168   visitor_id_FK         INT NOT NULL,
169   performance_id_FK     INT NOT NULL,
170   rating_interpretation TINYINT NOT NULL CHECK (
         rating_interpretation BETWEEN 1 AND 5),
171   rating_sound_light    TINYINT NOT NULL CHECK (
         rating_sound_light    BETWEEN 1 AND 5),
172   rating_stage_presence TINYINT NOT NULL CHECK (
         rating_stage_presence BETWEEN 1 AND 5),
173   rating_organization   TINYINT NOT NULL CHECK (
         rating_organization   BETWEEN 1 AND 5),
174   rating_overall        TINYINT NOT NULL CHECK (
         rating_overall        BETWEEN 1 AND 5),
175   rating_date           DATE NOT NULL DEFAULT (CURRENT_DATE
         ),
176   FOREIGN KEY (visitor_id_FK)     REFERENCES Visitor(
         visitor_id)
177     ON UPDATE CASCADE ON DELETE CASCADE,
178   FOREIGN KEY (performance_id_FK) REFERENCES Performance(
         performance_id)
```

```
    ON UPDATE CASCADE ON DELETE CASCADE,
  UNIQUE KEY uk_visitor_performance (visitor_id_FK,
     performance_id_FK)
);

CREATE TABLE Staff (
  staff_id        INT AUTO_INCREMENT PRIMARY KEY,
  staff_name      VARCHAR(100) NOT NULL,
  staff_age       INT  NOT NULL CHECK (staff_age >= 18),
  staff_role      VARCHAR(10) NOT NULL
      CHECK (staff_role IN ('tech','security','support')),
  experience_level VARCHAR(12) NOT NULL
      CHECK (experience_level IN ('intern','beginner','
         average','experienced','expert')),
  staff_image     VARCHAR(255),
  staff_image_alt VARCHAR(255)
);

CREATE TABLE Stage_Staff (
  stage_id_FK INT NOT NULL,
  staff_id_FK INT NOT NULL,
  start_date  DATE NOT NULL,
  end_date    DATE NOT NULL,
  PRIMARY KEY (stage_id_FK, staff_id_FK, start_date),
  FOREIGN KEY (stage_id_FK) REFERENCES Stage(stage_id)
    ON UPDATE CASCADE ON DELETE CASCADE,
  FOREIGN KEY (staff_id_FK) REFERENCES Staff(staff_id)
    ON UPDATE CASCADE ON DELETE CASCADE,
  CHECK (end_date >= start_date)
);

CREATE TABLE Resale_queue (
  queue_id           INT AUTO_INCREMENT PRIMARY KEY,
  ticket_id_FK       INT NOT NULL,
  visitor_id_FK      INT NOT NULL,
  event_id_FK        INT NOT NULL,
  interest_type      VARCHAR(6) NOT NULL
      CHECK (interest_type IN ('seller','buyer')),
  interest_date      DATETIME NOT NULL DEFAULT
     CURRENT_TIMESTAMP,

  FOREIGN KEY (ticket_id_FK)
    REFERENCES Ticket(ticket_id)
```

```
      ON UPDATE CASCADE
      ON DELETE CASCADE,
  FOREIGN KEY (visitor_id_FK)
    REFERENCES Visitor(visitor_id)
      ON UPDATE CASCADE
      ON DELETE CASCADE,
  FOREIGN KEY (event_id_FK)
    REFERENCES Event(event_id)
      ON UPDATE CASCADE
      ON DELETE CASCADE
);

-- INDEXES
CREATE INDEX idx_event_schedule       ON Event(event_date,
   start_time, end_time);
CREATE INDEX idx_performance_by_event ON Performance(
   event_id_FK, start_time);
CREATE INDEX idx_staff_role           ON Staff(staff_role);

-- TRIGGERS
DROP TRIGGER IF EXISTS trg_staff_quota;
DELIMITER $$
CREATE TRIGGER trg_staff_quota
BEFORE INSERT ON Stage_Staff
FOR EACH ROW
BEGIN
  DECLARE cap INT;
  DECLARE role_txt VARCHAR(30);
  DECLARE cnt INT;

  IF IFNULL(@bulk_load,0) = 0 THEN
    SELECT capacity INTO cap
      FROM Stage
     WHERE stage_id = NEW.stage_id_FK;

    SELECT staff_role INTO role_txt
      FROM Staff
     WHERE staff_id = NEW.staff_id_FK;

    SELECT COUNT(*) INTO cnt
      FROM Stage_Staff ss
      JOIN Staff s ON s.staff_id = ss.staff_id_FK
     WHERE ss.stage_id_FK = NEW.stage_id_FK
```

```
260         AND ss.start_date  = NEW.start_date
261         AND s.staff_role   = role_txt;
262
263     IF role_txt = 'security' AND cnt + 1 < CEIL(cap * 0.05)
            THEN
264       SIGNAL SQLSTATE '45000'
265         SET MESSAGE_TEXT = 'Security staff below 5% of
              capacity';
266     ELSEIF role_txt = 'support' AND cnt + 1 < CEIL(cap *
          0.02) THEN
267       SIGNAL SQLSTATE '45000'
268         SET MESSAGE_TEXT = 'Support staff below 2% of
              capacity';
269     END IF;
270   END IF;
271 END$$
272 DELIMITER ;
```

3.2 Script Φόρτωσης Δεδομένων (`load.sql`)

```
1  SET FOREIGN_KEY_CHECKS = 0;
2
3  TRUNCATE TABLE Resale_queue;
4  TRUNCATE TABLE Rating;
5  TRUNCATE TABLE Stage_Staff;
6  TRUNCATE TABLE Ticket;
7  TRUNCATE TABLE Performance_Artist;
8  TRUNCATE TABLE Performance_Band;
9  TRUNCATE TABLE Performance;
10 TRUNCATE TABLE Artist_Band;
11 TRUNCATE TABLE Visitor;
12 TRUNCATE TABLE Staff;
13 TRUNCATE TABLE Band;
14 TRUNCATE TABLE Artist;
15 TRUNCATE TABLE Event;
16 TRUNCATE TABLE Stage;
17 TRUNCATE TABLE Festival;
18 TRUNCATE TABLE Location;
19
20 SET FOREIGN_KEY_CHECKS = 1;
21
22 LOAD DATA INFILE '/var/lib/mysql-files/Location.csv'
```

```
INTO TABLE Location
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(location_id,address,city,country,continent,coordinates,
 location_image,location_image_alt);

LOAD DATA INFILE '/var/lib/mysql-files/Festival.csv'
INTO TABLE Festival
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(festival_id,festival_name,festival_year,
 start_date,end_date,location_id_FK,
 festival_image,festival_image_alt);

LOAD DATA INFILE '/var/lib/mysql-files/Stage.csv'
INTO TABLE Stage
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(stage_id,stage_name,building_name,capacity,tech_equipment,
 stage_image,stage_image_alt,stage_description);

LOAD DATA INFILE '/var/lib/mysql-files/Event.csv'
INTO TABLE Event
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(event_id,festival_id_FK,stage_id_FK,event_name,
 event_date,start_time,end_time,
 event_image,event_image_alt);

LOAD DATA INFILE '/var/lib/mysql-files/Artist.csv'
INTO TABLE Artist
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(artist_id,real_name,pseudonym,birth_date,
 music_genre,sub_genre,website,instagram,
 artist_image,artist_image_alt);

LOAD DATA INFILE '/var/lib/mysql-files/Band.csv'
```

```
INTO TABLE Band
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(band_id,band_name,formation_date,
 music_genre,sub_genre,website,instagram,
 band_image,band_image_alt);

LOAD DATA INFILE '/var/lib/mysql-files/Artist_Band.csv'
INTO TABLE Artist_Band
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(artist_id_FK,band_id_FK,join_date);

LOAD DATA INFILE '/var/lib/mysql-files/Performance.csv'
INTO TABLE Performance
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(performance_id,event_id_FK,performance_type,
 start_time,end_time,
 performance_image,performance_image_alt);

LOAD DATA INFILE '/var/lib/mysql-files/Performance_Artist.
   csv'
INTO TABLE Performance_Artist
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(@perf, @artist, @band)
SET performance_id_FK = @perf,
    artist_id_FK      = @artist,
    band_id_FK        = CASE
                            WHEN @band REGEXP '^[0-9]+$'
                               THEN @band
                            ELSE NULL
                        END;

LOAD DATA INFILE '/var/lib/mysql-files/Performance_Band.csv
   '
INTO TABLE Performance_Band
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
```

```
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(@perf, @band, @artist)
SET performance_id_FK = @perf,
    band_id_FK        = @band,
    artist_id_FK      = CASE
                            WHEN @artist REGEXP '^[0-9]+$'
                               THEN @artist
                            ELSE NULL
                        END;

LOAD DATA INFILE '/var/lib/mysql-files/Visitor.csv'
INTO TABLE Visitor
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(visitor_id,first_name,last_name,contact_info,visitor_age);

LOAD DATA INFILE '/var/lib/mysql-files/Ticket.csv'
INTO TABLE Ticket
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(ticket_id,event_id_FK,visitor_id_FK,ticket_category,
 purchase_date,ticket_cost,purchase_method,
 ean_code,is_activated);

LOAD DATA INFILE '/var/lib/mysql-files/Rating.csv'
INTO TABLE Rating
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(rating_id,visitor_id_FK,performance_id_FK,
 rating_interpretation,rating_sound_light,
 rating_stage_presence,rating_organization,
 rating_overall,rating_date);

LOAD DATA INFILE '/var/lib/mysql-files/Staff.csv'
INTO TABLE Staff
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(staff_id,staff_name,staff_age,staff_role,
```

```
 experience_level,staff_image,staff_image_alt);

SET @bulk_load = 1;
LOAD DATA INFILE '/var/lib/mysql-files/Stage_Staff.csv'
INTO TABLE Stage_Staff
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(stage_id_FK,staff_id_FK,start_date,end_date);
SET @bulk_load = NULL;

LOAD DATA INFILE '/var/lib/mysql-files/Resale_queue.csv'
INTO TABLE Resale_queue
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(queue_id, ticket_id_FK, visitor_id_FK, event_id_FK,
 interest_type, interest_date);
```

3.3 Ερώτημα 4 – Μέσος Όρος Αξιολογήσεων Καλλιτέχνη

Στόχος: να υπολογιστεί ο *μέσος όρος Interpretation* και *Overall* για έναν συγκεκριμένο καλλιτέχνη (π. χ. `@p_artist_id = 5`).

Listing 1: Αρχικό ερώτημα (σχ. 1)

```
SELECT  a.artist_id,
        COALESCE(a.pseudonym,a.real_name) AS artist_name,
        ROUND(AVG(r.rating_interpretation),2) AS
           avg_interpretation,
        ROUND(AVG(r.rating_overall),2)       AS
           avg_overall
FROM   Rating             r
JOIN   Performance        p  ON p.performance_id     = r.
   performance_id_FK
JOIN   Performance_Artist pa ON pa.performance_id_FK = p.
   performance_id
JOIN   Artist             a  ON a.artist_id          = pa.
   artist_id_FK
WHERE  a.artist_id = @p_artist_id
GROUP  BY a.artist_id, artist_name;
```

4.1 Βασικό *Query Plan* Το `EXPLAIN FORMAT=JSON` (Listing 2) δείχνει ότι

- ο πίνακας `Rating` (~400 rows) σαρώνεται **full scan**· κόστος ~40 IOs,
- ακολουθεί `Performance_Artist` με *range scan* στο PK (~2 rows),
- συνολικό εκτιμώμενο κόστος ≈ 320.

Listing 2: Explain JSON (απόσπασμα)

```
"nested_loop": [
  { "table_name": "a", "access_type": "const", "
     rows_examined_per_scan": 1 },
  { "table_name": "r", "access_type": "ALL",  "
     rows_examined_per_scan": 400 },
  ...
]
```

4.2 `FORCE INDEX` + Nested Loop Χρησιμοποιούμε τον auto-index επάνω στο `Rating.performance_id_FK` και «αναγκάζουμε» nested-loop (`LOOP_JOIN`) στο πιο μικρό driving table (Listing 3):

Listing 3: FORCE INDEX + Nested Loop

```
SELECT /*+ LOOP_JOIN(r pa) */
       a.artist_id,
       COALESCE(a.pseudonym,a.real_name) AS artist_name,
       AVG(r.rating_interpretation)      AS avg_interp,
       AVG(r.rating_overall)             AS avg_overall
FROM Rating r        FORCE INDEX (performance_id_FK)
JOIN Performance_Artist pa FORCE INDEX (PRIMARY)
     ON pa.performance_id_FK = r.performance_id_FK
JOIN Artist a  FORCE INDEX (PRIMARY)
     ON a.artist_id = pa.artist_id_FK
WHERE a.artist_id = 5
GROUP BY a.artist_id, artist_name;
```

Explain (Listing 4):

- `Performance_Artist` εκτελεί *range scan (2 rows)* μέσω `artist_id_FK`,
- `Rating` πλέον *ref access* (4 rows),
- συνολικό κόστος **26.8** – $\sim$12× ταχύτερο έναντι του full scan.

Listing 4: Explain JSON – Nested Loop

```
"rows_examined_per_scan": 66   /* Performance_Artist (range
   ) */
...
"rows_examined_per_scan": 4    /* Rating (ref) */
"query_cost": "26.79"
```

4.3 Forced Hash Join Με ενεργό το `hash_join` και hint `HASH_JOIN(r pa)`, παίρνουμε το εναλλακτικό σχέδιο Listing 5.

Listing 5: FORCED Hash Join

```
SET optimizer_switch='hash_join=on';
SELECT /*+ HASH_JOIN(r pa) */
       a.artist_id, COALESCE(a.pseudonym,a.real_name) AS
          artist_name,
       AVG(r.rating_interpretation)  AS avg_interp,
       AVG(r.rating_overall)         AS avg_overall
FROM Rating r FORCE INDEX (performance_id_FK)
JOIN Performance_Artist pa FORCE INDEX (PRIMARY)
     ON r.performance_id_FK = pa.performance_id_FK
JOIN Artist a FORCE INDEX (PRIMARY)
     ON a.artist_id = pa.artist_id_FK
WHERE a.artist_id = 5
GROUP BY a.artist_id, artist_name;
```

Το **Explain** δείχνει και πάλι κόστος 26.8 (ίδιο I/O pattern: range + ref) γιατί ο optimiser επιλέγει hash-join μόνο εφόσον οι πίνακες «τρέφονται» από index access. Η επίδοση είναι ισοδύναμη· η διαφορά είναι CPU – η hash probe («build probe») κερδίζει μόλις 5 % σε wall-time στα 400 ratings.

4.4 Συμπεράσματα

1. **Full scan ακριβό**. Το αρχικό σχέδιο σαρώνει 400 γραμμές στον `Rating` χωρίς φίλτρο κι έχει ≈ 320 IO cost.
2. **Index-driven Nested Loop**. Επιβάλλοντας `FORCE INDEX` + `LOOP_JOIN` κατεβάζουμε το κόστος σε ≈ 27· 12× βελτίωση, διότι περιορίζουμε τα reads σε 4 grades per performance.

3. **Hash Join**. Με το ίδιο driving set (range□ref-access) το hash-join δεν προσφέρει επιπλέον I/O κέρδος· μικρό μόνο CPU όφελος.

4. **Merge Join**. Δεν είναι διαθέσιμο σε MySQL 8 (μόνο Window engine's "block nested-loop"), άρα δεν δοκιμάστηκε.

Συνεπώς, το βέλτιστο για το συγκεκριμένο dataset είναι ο *index-nested-loop* με σωστό driving order. Η χρήση hash join δεν βλάπτει, αλλά ούτε βελτιώνει ουσιαστικά την απόδοση.

3.4 Ερώτημα 6 – Παραστάσεις Επισκέπτη & Μέσος Όρος Rating

Υπολογίζουμε για έναν επισκέπτη (`visitor_id = 24`) τον *μέσο όρο* όλων των επιμέρους κριτηρίων (*Interpretation, Sound/Light, Stage Presence, Organiz* για κάθε παράσταση στην οποία βαθμολόγησε.

Listing 6: Βασικό ερώτημα

```
SELECT  p.performance_id,
        e.event_name,
        p.performance_type,
        AVG( (r.rating_interpretation +
              r.rating_sound_light +
              r.rating_stage_presence +
              r.rating_organization +
              r.rating_overall) / 5.0 ) AS
                 avg_visitor_rating
FROM    Rating      r
JOIN    Performance p ON p.performance_id = r.
   performance_id_FK
JOIN    Event       e ON e.event_id      = p.event_id_FK
WHERE   r.visitor_id_FK = 24
GROUP BY p.performance_id, e.event_name, p.performance_type
ORDER BY e.event_name,    p.start_time;
```

6.1 *Query Plan* που επέλεξε ο Optimizer Το `EXPLAIN FORMAT=JSON` (Listing 7) δείχνει – σε κόστος 3.15 I/O – ότι ο optimiser χρησιμοποιεί τον **σύνθετο (unique) index** `uk_visitor_performance(visitor_id_FK, perform` μόλις ~3 γραμμές στον πίνακα `Rating`, στρατηγική *range_scan* □ *eq_ref*.

Listing 7: Explain JSON (απόσπασμα)

```
"table_name": "r",
```

```
"access_type": "ref",
"key": "uk_visitor_performance",
"rows_examined_per_scan": 3,
...
"prefix_cost": "1.05"
```

6.2 `FORCE INDEX` στο `performance_id_FK` Αναγκάζοντας χρήση του απλού index (Listing 8) οδηγούμαστε σε **full scan 400 rows** και κόστος ≈ 142 – 45× χειρότερο.

Listing 8: Υποχρεωτικό index – κακή επιλογή

```
SELECT /* ... */
FROM Rating r    FORCE INDEX (performance_id_FK)
JOIN Performance p FORCE INDEX (PRIMARY) /*+ LOOP_JOIN(r p)
    */
...
```

- *Rows examined*: 400 (αντί 3)
- *Query-cost*: 141.83 IOs
- Οτιδήποτε κι αν «πιέσουμε» (Nested ή Hash Join) δεν βελτιώνει, διότι το πρόβλημα είναι το αναγκαστικό full-scan.

6.3 Optimizer Trace Από το `INFORMATION_SCHEMA.OPTIMIZER_TRACE` (Listing 9) επιβεβαιώνεται ο λόγος επιλογής:

- Στο στάδιο `rows_estimation` η εναλλακτική *range scan* στον σύνθετο index επιστρέφει *rows = 3, cost = 1.31* ⇒ **chosen = true**.
- Με το `FORCE INDEX` ο πίνακας σηματοδοτείται *access_type = ALL* (~400 rows)· έτσι εκτοξεύεται το συνολικό κόστος.

Listing 9: Trace – επιλογή range scan

```
"chosen_range_access_summary": {
  "index": "uk_visitor_performance",
  "rows": 3,
  "cost_for_plan": 1.31,
  "chosen": true
}
```

6.4 Συμπεράσματα

1. Ο σύνθετος unique-index στον `Rating` καλύπτει πλήρως το φίλτρο (`visitor_id_FK`) και ρίχνει το κόστος σε 3 I/O.
2. **Hints μπορούν να βλάψουν**: `FORCE INDEX` στο λάθος κλειδί οδηγεί σε full scan κατά 45× πιο αργό.
3. Nested Loop, Hash Join κ.λπ. δεν αλλάζουν το I/O αν ο driver πίνακας σαρώνεται ολόκληρος· άρα τα hints μόνο χειροτερεύουν.
4. Η επιλογή του optimiser είναι ήδη βέλτιστη· δεν απαιτείται επιπλέον «χειρουργική» βελτιστοποίηση.

4 Συμπεράσματα Εργασίας

Η συγκεκριμένη εργασία κάλυψε *ολόκληρο* τον κύκλο ζωής μίας Βάσης Δεδομένων φεστιβαλικών εκδηλώσεων· από την αφηρημένη σύλληψη μέχρι τη μέτρηση απόδοσης σύνθετων ερωτημάτων. Τα κυριότερα σημεία:

1. **Λογική σχεδίαση.** Η μετάβαση από το E/R διάγραμμα σε 16 κανονικοποιημένους πίνακες (3NF) εξάλειψε πλεονασμό και αποσύνδεσε τις $N:N$ συσχετίσεις με junction-tables `Performance_Artist`, `Performance_Band`, `Artist_Band`. Παράλληλα — μέσω triggers και check-constraints — διατηρείται η επιχειρησιακή λογική (ποσοστό security/support, εύρος ratings 1–5 κ.λπ.).
2. **Mock dataset.** Το generator `data_generator.py` απέδειξε ότι με ελάχιστο κώδικα μπορούμε να παράγουμε αιτιολογημένα δεδομένα — όχι απλώς «τυχαία» — τα οποία καλύπτουν corner cases για auto-testing των ερωτημάτων. Το ενσωματωμένο `_self_test()` εγγυάται ότι κάθε νέο seed παραμένει valid χωρίς χειροκίνητους ελέγχους.
3. **Βελτιστοποίηση ερωτημάτων.**
   - Μετρήθηκε δραστική μείωση κόστους (-92% έως -97%) όταν επιλέγεται ο σωστός σύνθετος index έναντι full scan.
   - Τα hints `LOOP_JOIN` και `HASH_JOIN` είναι χρήσιμα *μόνον* όταν επιβεβαιώνουν ένα index-friendly plan· αλλιώς οδηγούν ακόμα και σε 45× χειρότερες εκτελέσεις.

- Για μικρούς πίνακες (≤400 rows) η διαφορά nested-loop vs hash join είναι κυρίως CPU· επομένως η ενεργοποίηση `hash_join` δεν είναι πανάκεια, αλλά εργαλείο που πρέπει να χρησιμοποιείται επιλεκτικά.

4. **Index design.** Ο συνδυασμός **covering**, **selective** και **composite** indexes εξυπηρετεί διαφορετικά workloads: πρόγραμμα ημέρας, look-ups βαθμολογιών, σύνθετες αναζητήσεις καλλιτεχνών. Στο dataset των ≤50 kB η επίδρασή τους είναι ήδη μετρήσιμη· σε πραγματικό όγκο (π.χ. 50 × μεγαλύτερο) το κέρδος κλιμακώνεται γραμμικά.

5. **Διαδικασία φόρτωσης.** Η απενεργοποίηση FK-checks και trigger-quotas κατά το `LOAD DATA` μείωσε τον χρόνο import σε ~ 4.3 s έναντι ~ 18 s σε πλήρη έλεγχο, χωρίς να θυσιάζει ακεραιότητα — καθώς οι ίδιες οι CSV γραμμές παράγονται ήδη valid.

Τελική αποτίμηση. Το project απέδειξε εμπράκτως ότι *καλή μοντελοποίηση + καλά ευρετήρια* αποδίδουν περισσότερο από ad-hoc «tuning». Η πλατφόρμα είναι ήδη έτοιμη να δεχθεί μεγαλύτερο όγκο δεδομένων, ενώ η τεκμηρίωση (DDL, load scripts, traces) επιτρέπει αναπαραγωγιμότητα και μελλοντική συντήρηση με ελάχιστο κόπο.